

CIFAR-10 CNN Deployment Guide

Project Structure

```
cifar10-classifier/
├── app.py           # Flask application
├── train.py         # Model training script
├── models/
│   └── cifar10_cnn.pth # Trained model weights
├── templates/
│   └── index.html    # Web interface
├── requirements.txt # Python dependencies
├── Procfile         # Heroku process file
├── runtime.txt      # Python version
└── README.md        # Documentation
```

Step 1: Train the Model

First, train the CNN model locally:

```
bash

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install torch torchvision numpy

# Train the model
python train.py
```

This will:

- Download the CIFAR-10 dataset
- Train a CNN for 50 epochs
- Save the best model to `models/cifar10_cnn.pth`
- Achieve ~85-90% accuracy on test set

Training takes about 30-60 minutes on CPU (faster with GPU).

Step 2: Test Locally

Test the Flask app locally before deployment:

```
bash

# Install all dependencies
pip install -r requirements.txt

# Run the Flask app
python app.py
```

Visit <http://localhost:5000> to test the web interface.

Step 3: Deploy to Heroku

Prerequisites

- Heroku account (free tier works)
- Heroku CLI installed
- Git installed

Deployment Steps

```
bash

# Login to Heroku
heroku login

# Create a new Heroku app
heroku create your-app-name

# Add buildpack for PyTorch (reduces slug size)
heroku buildpacks:add --index 1 heroku/python

# Initialize git repository
git init
git add .
git commit -m "Initial commit"

# Deploy to Heroku
git push heroku main

# Open your app
heroku open
```

Important Notes for Heroku

1. Model Size Optimization: The trained model file is ~45MB. Heroku has a 500MB slug size limit. If needed, use model compression:

```
python

# Add to train.py after training
import torch
model_scripted = torch.jit.script(model)
model_scripted.save('models/cifar10_cnn_optimized.pt')
```

2. Memory Management: Free Heroku dynos have 512MB RAM. The app is optimized to run on CPU with minimal memory:

- Uses CPU inference only
- Batch size of 1 for predictions
- No unnecessary caching

3. Cold Starts: Free dynos sleep after 30 minutes of inactivity. First request may take 10-15 seconds.

4. Environment Variables: Set if needed:

```
bash

heroku config:set FLASK_ENV=production
heroku config:set MAX_CONTENT_LENGTH=16777216 # 16MB file upload limit
```

Alternative Deployment Options

1. Google Cloud Run

```
bash

# Build container
docker build -t gcr.io/PROJECT_ID/cifar10-classifier .

# Push to registry
docker push gcr.io/PROJECT_ID/cifar10-classifier

# Deploy
gcloud run deploy --image gcr.io/PROJECT_ID/cifar10-classifier --platform managed
```

2. AWS Elastic Beanstalk

```
bash
```

```
# Install EB CLI
```

```
pip install awsebcli
```

```
# Initialize and deploy
```

```
eb init -p python-3.10 cifar10-classifier
```

```
eb create cifar10-env
```

```
eb open
```

3. Railway.app

- Connect GitHub repository
- Auto-detects Python app
- Deploys automatically
- More generous free tier than Heroku

Performance Optimization

Model Optimization

```
python
```

```
# Quantization (reduces model size by 4x)
```

```
import torch.quantization
```

```
model_quantized = torch.quantization.quantize_dynamic(  
    model, {torch.nn.Linear}, dtype=torch.qint8  
)
```

Caching Predictions

Add Redis for caching frequent predictions:

```
python
```

```
import redis
```

```
cache = redis.Redis(host='localhost', port=6379)
```

Load Balancing

For high traffic, use:

- Multiple Heroku dynos
- CDN for static assets
- Database for logging predictions

Monitoring

Heroku Logs

```
bash

heroku logs --tail
```

Add Monitoring

```
python

# Add to app.py
import logging
logging.basicConfig(level=logging.INFO)

@app.route('/predict', methods=['POST'])
def predict_route():
    logging.info(f'Prediction request from {request.remote_addr}')
    # ... rest of code
```

Troubleshooting

Issue: Model not found

Solution: Ensure `models/cifar10_cnn.pth` is included in git:

```
bash

git add -f models/cifar10_cnn.pth
```

Issue: Memory error on Heroku

Solution: Use model quantization or upgrade to paid dyno

Issue: Slow predictions

Solution:

- Use CPU-optimized PyTorch build
- Reduce model complexity
- Add prediction caching

Issue: Upload fails

Solution: Check file size limits in `app.py`:

```
python
```

```
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB
```

Security Considerations

1. Input Validation:

- Verify file types
- Limit file sizes
- Sanitize filenames

2. Rate Limiting:

```
python
```

```
from flask_limiter import Limiter  
limiter = Limiter(app, default_limits=["100 per hour"])
```

3. HTTPS:

- Heroku provides free SSL
- Enforce HTTPS in production

4. CORS (if building API):

```
python
```

```
from flask_cors import CORS  
CORS(app, resources={r"/predict": {"origins": "your-domain.com"}})
```

Cost Estimates

Heroku

- **Free:** 550 dyno hours/month (good for demos)
- **Hobby (\$7/mo):** Always-on, custom domains
- **Standard (\$25/mo):** More RAM, better for production

Alternatives

- **Railway:** \$5/mo for 512MB RAM
- **Render:** Free tier with auto-sleep

- **Google Cloud Run:** Pay per request, very cheap for low traffic

Next Steps

1. Improve Model:

- Train for more epochs
- Use data augmentation
- Try transfer learning (ResNet, EfficientNet)

2. Add Features:

- Batch predictions
- Model explainability (Grad-CAM)
- User feedback collection
- Prediction history

3. Scale:

- Add database for logging
- Implement A/B testing
- Set up CI/CD pipeline
- Add analytics dashboard

Resources

- [Heroku Python Docs](#)
- [PyTorch Deployment](#)
- [Flask Production Best Practices](#)

Support

For issues or questions:

- Check Heroku logs: `heroku logs --tail`
- Review Flask error messages
- Test locally first before deploying